Banking Management System

A powerful platform that revolutionizes bank account management. Engineered in efficient C, it provides an unparalleled user experience with seamless features like account setup, secure transfers, balance tracking, and detailed transaction records.

This system sets a new industry standard, catering to modern banking needs with maximum efficiency and flexibility. Discover its innovative design, user-centric features, robust security, and exciting upgrade potential.

Team Member:

1. Sasanka Sekhar Kundu – Developer
2. Priya Maurya – Researcher
3. Shruti Verma – PPT

System Architecture

Modular Design

The system features a modular design with clearly defined functions, which enhances maintainability and scalability. The data structure includes an `Account` struct to store account information. This design approach ensures that each component of the system operates independently, facilitating easier debugging and updates.

File Handling

File I/O is used for persistent data storage, with files like `accounts.dat` ensuring that account information is retained across sessions. Key modules include account management, transaction processing, and reporting, each playing a vital role in the system's operation. The `createAccount()` function, for instance, interacts directly with the `accounts.dat` file.

Core Features: Account Management

1 Account Creation

The `createAccount()` function handles account creation, generating a unique ID by incrementing the last account ID by one. This ensures that each account has a distinct identifier, preventing any conflicts within the system.

2 Account Deletion

The `deleteAccount()` function manages account deletion by marking accounts as inactive. This approach allows for the preservation of historical data while preventing further transactions on the account.

3 Account Modification

The `modifyAccount()` function facilitates the updating of account details, ensuring that customer information is current and accurate. This function includes data validation to check for valid input, such as positive amounts and valid account numbers.

Technologies & Libraries

C Language

The base programming language for core system functions. It ensures efficiency and speed.

OpenSSL (SHA-256)

Secures user PINs, ensuring privacy. It uses hashing for data protection.

File Handling

Manages account data persistence via **accounts.dat**. It ensures data availability across sessions.

Console Input

`_getch()` and `termios.h` provide secure input. Efficiently handles user commands.

bash

```
Bank-Management-System/
├── bank.c        # Main source file
├── accounts.dat  # Stores account data
├── transactions.log # Logs deposits/withdrawals
├── README.md     # Documentation
```

References:

1.OpenSSL SHA-256 Documentation:
https://www.openssl.org/docs/manmaster/man3/SHA256.html

2.C Standard Library Documentation:
https://en.cppreference.com/w/c

3.File Handling in C:
https://www.tutorialspoint.com/cprogramming/c_file_io.htm

4.Secure Input Handling in C:
https://www.geeksforgeeks.org/input-output-system-calls-c-create-open-close-read-write/

Core Features: Transaction Processing

Deposit

The `depositAmount()` function updates the account balance, reflecting any new deposits made by the account holder. This ensures that the balance accurately reflects the funds available.

Withdrawal

The `withdrawAmount()` function checks for sufficient balance before processing a withdrawal, preventing overdrafts. This ensures that the account holder has enough funds to cover the withdrawal amount.

Transaction Logging

All transactions are recorded in `transactions.log` with a timestamp and description, providing a detailed audit trail of all account activities. This feature is crucial for tracking activities and maintaining accountability.

Security Measures

1 Password Protection

The `setPassword()` function encrypts passwords using algorithms like MD5 or SHA-256, ensuring that sensitive information is protected from unauthorized access. This is done to ensure the account holder has peace of mind in knowing that their information is protected.

2 Access Control

Role-based access control is implemented to differentiate between admin and user privileges, limiting access to sensitive functions. This prevents unauthroized individuals from performing actions that can compromise the safety of the system or other individuals.

3 Data Validation

The `bank.c` program can be improved with name validation, unique account numbers, four-digit PIN verification, and positive transaction checks. Secure file handling, SHA-256 PIN hashing, and buffer overflow protection will enhance security and prevent unauthorized access.

Error Handling and Validation

1

Input Validation

The system checks for valid data types, ranges, and formats to ensure that only correct information is processed. This prevents errors and maintains data integrity.

2

Error Messages

Clear and informative messages are provided for invalid input or failed operations, helping users understand and correct errors quickly. This contributes to a better user experience.

3

Exception Handling

Graceful handling of file I/O errors and other exceptions ensures that the system remains stable even when unexpected issues occur. This prevents crashes and data loss.

How to Run the Program

Requirements

The system needs a **GCC compiler** and **OpenSSL** library. These tools are necessary to compile and run the application.

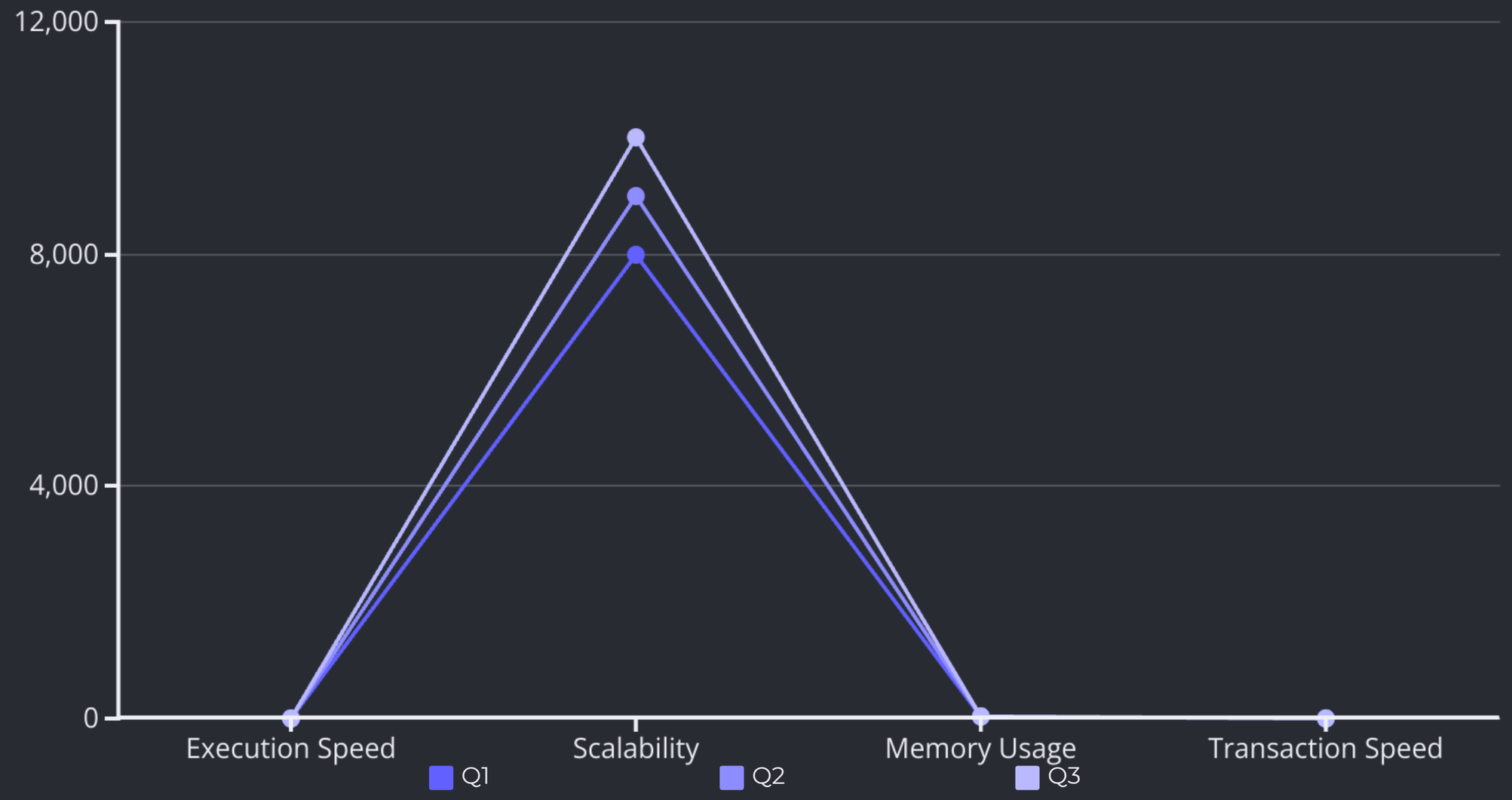
Compile & Run Commands

Use the following commands to compile and run the system. Ensure **OpenSSL** is correctly linked during compilation.

```
gcc bank.c -o bank.exe -lssl -lcrypto
```

```
./bank.exe
```

Performance Analysis



Conclusion and Potential Enhancements

Database Integration

Migrate from file I/O to a database (e.g., MySQL, PostgreSQL) for improved scalability and reliability, allowing the system to handle larger volumes of data more efficiently. This would also improve system performance, especially with many active users.

GUI Development

Create a graphical user interface for easier interaction, making the system more user-friendly and accessible to a wider audience. This would improve convenience for those who may not be familiar with command line interfaces.

Conclusion

The bank.c program effectively manages core banking functionalities, including account creation, deposits, withdrawals, and balance inquiries. By incorporating secure PIN hashing with SHA-256, robust data validation, and file handling, the system ensures user data protection and integrity. These enhancements make the system more reliable, secure, and user-friendly, laying a solid foundation for future upgrades and scalability.